Multi-objective drift-aware MLOps agent for model retraining

Kumuda Valli V
Computer Science
California State University, Fullerton
Fullerton, United States of America
vk.valli2909@gmail.com

Anli Ji
Computer Science
California State University, Fullerton
Fullerton, United States of America
anli.ji@fullerton.edu

***Abstract*—Deployed ML models face non-stationary data and delayed labels, yet most retraining strategies rely on fixed schedules or brittle drift thresholds that ignore operational cost. In this setting, retraining is a sequential decision problem rather than a one-time threshold choice. We present *DARE* (Drift-Aware Retraining Engine), an offline, event-driven retraining framework that separates ingestion, monitoring, policy decisions, training, and logging through explicit schemas and deterministic replay.**

We compare DARE against practical heuristic retraining baselines, including fixed-interval, threshold-persistence, and action-rate-matched fixed scheduling policies. On Airline, the locked LinUCB policy achieves a higher mean F1 while substantially reducing intervention frequency relative to heuristic and action-rate matched baselines. Additional runtime-calibrated evaluation shows that these gains persist when measured using the measured cumulative update wall-clock time. These results suggest that conservative sequential retraining policies can improve performance while reducing unnecessary operational overhead under delayed feedback.

***Index Terms*—Concept Drift, MLOps, Multi-Objective Optimization, Contextual Bandits, Delayed Feedback, Operationally Efficient Model Retraining**

I. Introduction

Deployed machine learning models degrade when data distributions shift over time, and retraining too frequently is expensive in compute, engineering effort, and operational risk [1], [2]. In production, teams often use fixed schedules or static drift thresholds to trigger updates. These heuristics are simple but brittle: a setting that works in one regime can over-trigger or under-trigger in another [3], [4].

This work treats retraining as a sequential decision problem with explicit operational constraints. Instead of asking whether drift crossed a threshold at one instant, we ask which action should be taken now (*defer* or *fine-tune*) given current drift/performance signals and expected operational overhead, while accounting for delayed feedback [5], [6]. This framing is motivated by recent ML systems work showing that retraining policy quality depends on balancing utility and pipeline operational overhead, not detection alone [7], [8], [9].

In this paper, *MLOps* (Machine Learning Operations) refers to the engineering practice of deploying, monitoring, and maintaining machine learning systems in production. We use *events* to denote timestamped records emitted by pipeline stages (e.g., monitoring outputs and execution outcomes). A *policy* is a decision rule that maps the current system state to an *action*, where actions in our setting include `defer` and `fine_tune`.

We implement this framing in *DARE*, an offline event-driven architecture with deterministic replay and schema-validated event contracts. The architecture decouples monitoring from decision and execution, enabling reproducible, policy-level comparisons under matched data windows and delay conditions.

We evaluate DARE on temporally ordered benchmarks with delayed feedback and compare it against common heuristic retraining strategies, including an action-rate-matched fixed-interval baseline designed to separate policy quality from update frequency. The goal of this evaluation is to test whether a sequential, operationally efficient retraining policy can maintain task performance while reducing unnecessary interventions under non-stationary data.

In summary, this paper makes three contributions: (1) an operational formulation of retraining as a runtime-constrained sequential decision problem under delayed labels; (2) a reproducible event-driven evaluation framework for policy comparison; and (3) evidence that conservative contextual-bandit retraining outperforms practical heuristic retraining strategies, including an action-rate-matched fixed schedule, while reducing measured execution overhead (wall-clock runtime) under delayed feedback.

The remainder of the paper presents related work, system design, the experimental protocol, and results focused on Airline (a temporally ordered flight-delay classification stream), with Elec2 (an electricity market binary classification benchmark) and delayed-label stress tests for calibration and robustness.

II. Related Work

Drift detection and monitoring. The concept-drift literature has established robust taxonomies and detection mechanisms, ranging from classical surveys to recent systems work. Dong *et al.* [7] and Böther *et al.* [8] highlight systems-level considerations in drift-aware pipelines, including orchestration and data-centric controls. Katalay *et al.* [9] combine multiple monitoring criteria (e.g., distribution and divergence metrics) for automated retraining decisions. However, these approaches primarily improve *detection* or trigger design; they do not frame retraining as a sequential decision policy that explicitly trades

1946-0759/26/$31.00 ©2026 IEEE
DOI 10.1109/ICMLA69890.2026.00145

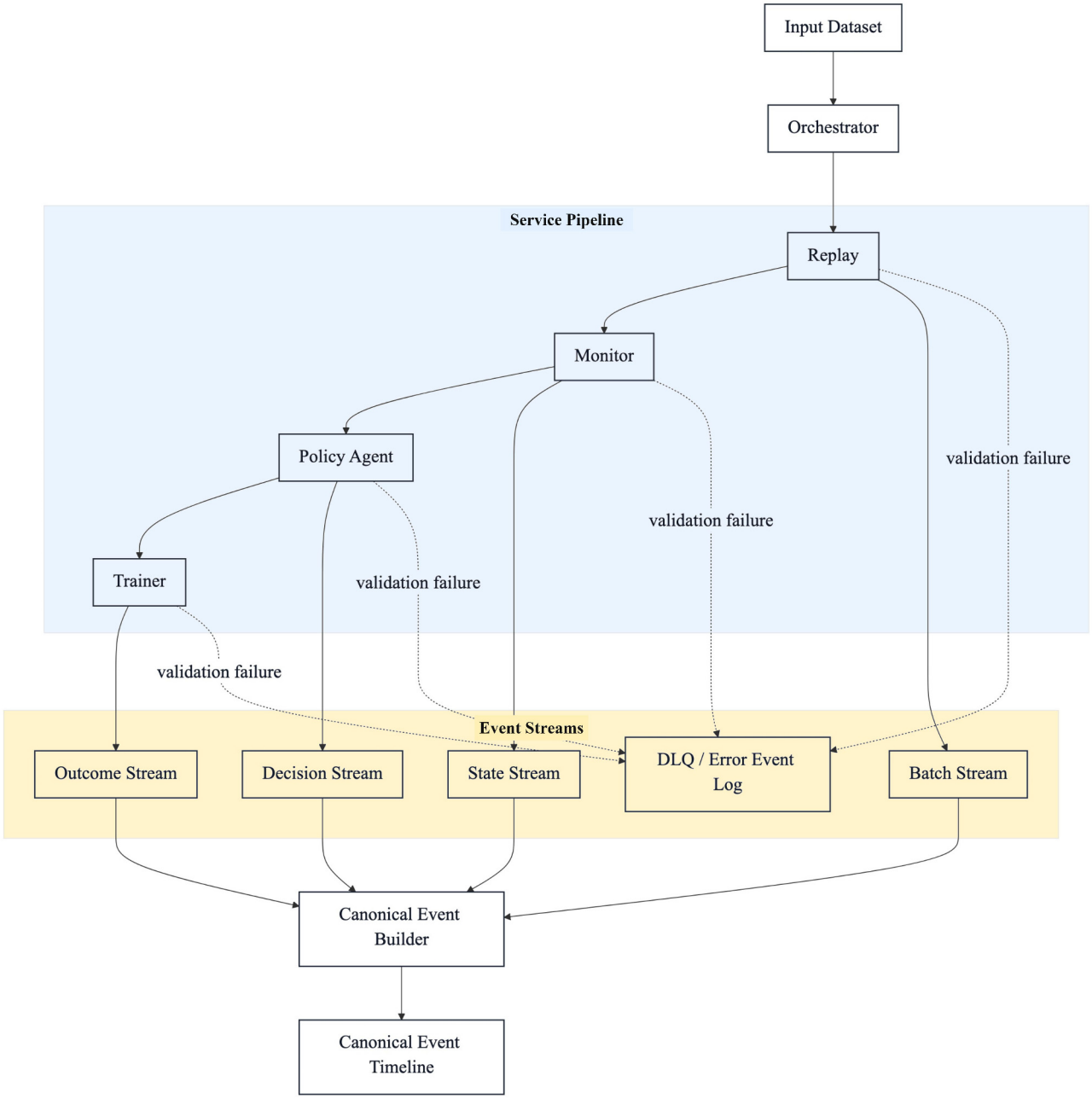


Fig. 1. DARE event-driven pipeline. Dashed arrows indicate validation-failure routing to the DLQ.

off performance and intervention cost under delayed labels. Although contextual bandit methods provide a sequential decision mechanism [12], they are typically studied for recommendation-style action selection rather than retraining-control decisions under drift, delayed labels, and explicit operational overhead.

Retraining policy and operational cost. Chislett *et al.* [2] propose AMUSE, an adaptive model-updating approach that learns retraining timing in a simulated drift environment while balancing predictive utility and operational overhead. Unlike fixed schedules or static trigger rules, AMUSE frames update timing as a learned sequential policy, though its evaluation is primarily simulation-based. Regol *et al.* [3] further analyze retraining timing and emphasize sensitivity to operating assumptions. Systems such as Modyn further highlight that the quality of retraining depends on data/pipeline choices and operational overhead, not just model metrics [8]. Still, most methods rely on fixed schedules, threshold rules, or static selection logic rather than learning a conservative action policy

over time that is explicit about the action rate and budget. Action rate is the fraction of decision windows in which the policy triggers a model update.

Sequential decision methods under delayed feedback. Rank *et al.* [5] and Moothedath [6] motivate sequential decision-making under shifting environments and delayed feedback, but do not provide an end-to-end event-driven retraining orchestration framework with deterministic replay. In parallel, fairness-under-drift work shows that subgroup behavior can shift over time and should be monitored continuously [10]. Yet these lines are typically theoretical or domain-specific and are not integrated into an end-to-end retraining orchestration pipeline with deterministic replay, schema-level traceability, and direct comparison against practical threshold baselines.

Positioning. Our work connects these strands by evaluating a conservative contextual-bandit retraining policy in an MLOps-oriented, event-driven framework, with explicit operationally efficient decisions and delayed-label robustness testing against standard heuristic retraining policies.

III. System Design

A. Design Objectives

DARE is designed for policy-level retraining decisions under three practical constraints: non-stationary inputs, delayed labels, and bounded operational resources. The core objective is to compare retraining policies under a shared execution protocol, not to optimize drift detection in isolation [3], [4], [7], [9]. Following this objective, the system enforces strict separation between (i) monitoring signals, (ii) policy decisions, and (iii) execution outcomes. Strict separation is enforced at the interface level: monitor emits state only, policy emits action only, and trainer emits outcome only. No component mutates another component's artifacts in place; communication is event-based and append-only. This prevents signal leakage (e.g., future outcome information entering policy state), isolates failures to component boundaries, and ensures that policy comparisons reflect decision logic rather than hidden coupling between monitoring and execution stages.

B. Event-Driven Runtime

The implemented runtime is an offline, event-driven pipeline:

```
replay → monitor → policy →
trainer → canonical_log
```

Replay emits time-ordered batch and delayed-label events from CSV windows. Monitor consumes these events and emits state events. Policy consumes states and emits actions. Trainer consumes actions and emits cost/model-update outcomes. All stages write immutable JSON Lines (JSONL) events, which are merged into an append-only, canonical timeline for reproducible replay and post-hoc analysis [11]. This architecture follows the same systems motivation as recent MLOps pipeline work while making decision traces explicit [8], [9].

C. State, Action, and Outcome Contracts

Each decision step is represented by a compact state record

$$s_t = [\text{drift features},\ \text{performance features},\ \text{fairness features},\ \text{readiness flags},\ \text{metadata}] \tag{1}$$

The implementation uses a schema-validated canonical state object (including IDs, window metadata, and model versioning), but policy decisions in this paper rely primarily on drift/performance readiness signals.

In the current implementation, `drift_core` is the primary trigger signal; fairness fields are logged and available to policies, but are not required in the locked Airline operating point. Here, "locked" means a fixed, pre-declared evaluation configuration (reward weights, action semantics, exploration settings, batching, and delay protocol) applied unchanged across all six seeds and baseline comparisons, with no per-run retuning.

Action semantics are intent-level:

$$a_t \in \{\texttt{defer},\ \texttt{fine_tune}\}$$

for the LinUCB policy path. The execution layer also supports `full_retrain` for heuristic control policies. For the locked Airline comparison, all updating policies execute the same `fine_tune` operation. The `full_retrain` action is implemented for extensibility and heuristic evaluation but is not used in the reported Airline operating point. Consequently, differences between policies reflect update timing rather than different update semantics. Outcomes are recorded through cost events (operational overhead, time, energy, pre-update metrics, fairness diagnostics) and optional model-update events when an update is executed.

D. Policy Layer

We evaluate three deterministic baselines: no retrain, fixed interval, and threshold persistence, against one learned policy (LinUCB) [12].

The learned policy is conservative by design in three ways: (i) the action space is restricted to $\{\texttt{defer}, \texttt{fine_tune}\}$ (no aggressive full retrain action in the main RL path), (ii) reward shaping penalizes intervention cost and update frequency, and (iii) exploration is bounded by a fixed schedule to avoid unstable action-rate spikes. Given context vector x_t, LinUCB selects

$$a_t = \arg\max_{a \in \mathcal{A}} \left(\hat{\theta}_a^\top x_t + \alpha \sqrt{x_t^\top A_a^{-1} x_t} \right), \tag{2}$$

where α controls exploration. We lock $\alpha = 0.25$ and fix the exploration schedule across all runs to avoid run-specific retuning.

E. Reward Construction with Delayed Feedback

Rewards are computed from logged outcomes, not from future information at decision time. Let $\Delta\tilde{p}_t$ denote normalized rolling performance change and $\tilde{c}_t$ normalized measured runtime. The base reward is

$$\begin{aligned} r_t = \Delta\tilde{p}_t - w_{\text{cost}}\tilde{c}_t - \lambda_{\text{act}}\mathbf{1}[a_t = \texttt{fine_tune}] \\ + \lambda_{\text{def}}\mathbf{1}[a_t = \texttt{defer} \wedge d_t < \tau_d]. \end{aligned} \tag{3}$$

DARE is metric-agnostic at the orchestration level: the runtime, event contracts, and policy loop remain unchanged across tasks, while only the performance term $\Delta\tilde{p}_t$ is task-specific. In this paper, Airline uses rolling ΔF1 (imbalanced classification), whereas Elec2 uses rolling ΔAccuracy. This design allows the same decision framework to transfer across tasks with different primary performance metrics. An optional fairness penalty term is implemented but not enforced in the locked main result:

$$r_t^{\text{final}} = r_t^{\text{base}} - w_{\text{fair}}\tilde{g}_t \tag{4}$$

This keeps fairness measurable without over-claiming fairness-optimized control in the present phase [10].

F. Determinism and Reproducibility Controls

DARE enforces deterministic event IDs, schema validation, and dead-letter queue (DLQ) routing for invalid payloads. Canonical logs are append-only, and replay is file-order deterministic. Trainer execution is idempotent through action-level processed-index checks. These controls ensure that policy comparisons are attributable to policy logic and configuration, rather than orchestration artifacts.

IV. Experiments

A. Experimental Questions

We evaluate four questions: (1) Can a conservative contextual-bandit retraining policy reduce operational overhead relative to standard heuristic triggers? (2) Does this reduction preserve or improve task performance? (3) How stable are outcomes across random seeds? (4) Does policy behavior remain stable under delayed-label feedback?

B. Dataset

Our primary benchmark is Airline 120k, a temporally ordered binary classification stream for predicting arrival delay > 15 minutes [13]. The stream is class-imbalanced (positive class frequency $\approx 17.5\%$), so we report F1 alongside balanced accuracy. Elec2 is a temporally ordered binary stream benchmark derived from electricity market signals [14], [15]. In our calibration runs, Elec2 is used as secondary calibration evidence rather than as a headline-claim-setting benchmark. Data are processed in fixed batches of 200 for all compared policies.

C. Setup and Protocol

We compare three deterministic baselines (*no retrain*, *fixed interval*, and *threshold persistence*) against a locked LinUCB policy with action space $\{\texttt{defer}, \texttt{fine_tune}\}$ [3], [4]. In the reported Airline experiments, all updating policies execute the same `fine_tune` operation. Consequently, observed differences arise from update timing and frequency rather than different retraining mechanisms. The no-retrain baseline never updates the model after initialization. These baselines are intended to represent simple operational policies commonly used when learned retraining control is unavailable.

To distinguish adaptive decision quality from simply updating less frequently, we additionally evaluate an action-rate-matched fixed-interval baseline whose interval is selected solely to match the locked LinUCB intervention frequency before evaluation. All reward parameters, exploration settings, and baseline hyperparameters are fixed prior to the six-seed evaluation.

For Airline, threshold persistence uses drift threshold $T = 1.8$ and persistence $K = 3$: retraining is triggered only when `drift_core` exceeds T for K consecutive decision windows; otherwise the policy defers. The threshold-persistence hyperparameters ($T = 1.8$, $K = 3$) were selected through our own predefined baseline calibration on development runs (separate from the final six-seed comparison table). We evaluated candidate settings by their stability–coverage behavior (avoiding degenerate always-update/never-update patterns) and then froze the selected pair before final policy comparison to avoid outcome chasing.

The underlying Airline predictor is an incremental linear classifier implemented with `SGDClassifier` using logistic loss. Before each eligible update, the current model is evaluated on the incoming labeled window; the reported performance metrics therefore describe pre-update behavior and avoid evaluating the model on a window after training on it. An executed `fine_tune` applies one `partial_fit` update to that window using inverse-frequency class weights and then stores the resulting model as a new version. If delayed labels are unavailable, the requested update is not executed. This design isolates policy quality from differences in model architecture or optimization procedure. All policies therefore share identical model architecture, feature representation, windowing protocol, and delayed-label configuration; only the retraining decision policy differs. All compared policies use this identical predictor, feature pipeline, and update procedure, so their only experimental difference is when they request an update.

D. Metrics

We report cumulative update runtime, F1, balanced accuracy, action rate, and update count. Runtime captures the operational overhead directly controlled by the retraining policy, measured via wall-clock execution time rather than fixed proxy-cost assumptions, while F1 and balanced accuracy measure predictive performance under class imbalance. Action rate and update count quantify intervention aggressiveness. While balanced accuracy remains close to chance for all policies, indicating the underlying classifier has limited overall discriminative power on this task, the relative precision/recall differences across retraining policies (Table I) still meaningfully distinguish policy quality, since all policies share the identical underlying model and differ only in retraining timing.

Runtime accounting. We measure cumulative wall-clock update runtime for each retraining action executed under a fixed hardware and software configuration. Because all compared policies execute the same update routine, cumulative runtime provides a directly observed measure of operational overhead. Under usage-based cloud pricing, infrastructure

TABLE I
AIRLINE 120K POLICY COMPARISON (6 SEEDS, MEAN ± SD)

Policy	Runtime(s)	F1	Precision	Recall	Action rate	Balanced accuracy
No retrain	0.0000±0.0000	0.0000±0.0000	0.0000±0.0000	0.0000±0.0000	0.0000±0.0000	0.0000±0.0000
Fixed (N=10)	2.2936±1.3266	0.1724±0.0114	0.1309±0.0076	0.4800±0.0431	0.0991±0.0000	0.5089±0.0033
Fixed matched (N=64)	0.3870±0.2118	0.1742±0.0278	0.1258±0.0182	0.4990±0.0901	0.0154±0.0000	0.5026±0.0053
Threshold persistence (1.8,3)	1.8466±0.2802	0.1547±0.0543	0.1167±0.0290	0.4465±0.1993	0.1145±0.0000	0.5002±0.0041
LinUCB (locked)	0.2687±0.0859	0.2065±0.0254	0.1596±0.0179	0.6012±0.1310	0.0157±0.0036	0.5057±0.0059

TABLE II
DELAYED-LABEL STRESS TEST (AIRLINE, LINUCB LOCKED; MEANS OVER 6 SEEDS)

Delay	Act. rate	Updates	F1	Reward
0	0.0157	9.17	0.2058	-0.0010
2	0.0157	9.17	0.2065	-0.0010
5	0.0157	9.17	0.2076	-0.0010
10	0.0157	8.17	0.2229	0.0015

TABLE III
METRIC ALIGNMENT ABLATION (AIRLINE, LINUCB; MEANS OVER 6 SEEDS)

Reward metric	Action rate	Updates	F1
ΔAccuracy	0.2886	168.83	0.1813
ΔF1	0.0143	8.33	0.2290

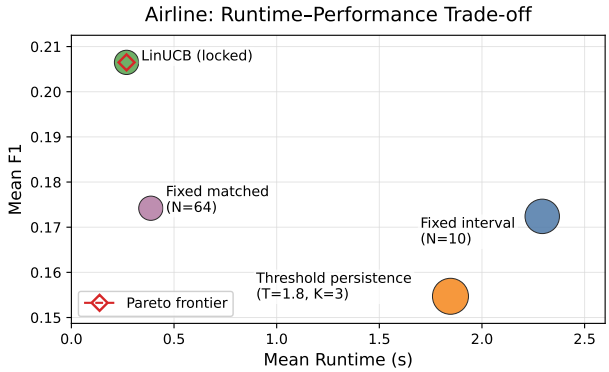


Fig. 2. Airline Pareto trade-off (cumulative update runtime vs. F1). Each bubble represents a retraining policy, with bubble area proportional to action rate; larger bubbles indicate more frequent update decisions. The red diamond marks the non-dominated Pareto-front policy. LinUCB occupies the favorable low-runtime/high-F1 region.

costs scale approximately with compute time for a fixed deployment configuration; runtime therefore serves as the primary experimental resource metric while avoiding provider-specific billing assumptions.

Monitoring, drift computation, and state construction are executed for every evaluated policy. Because these operations are identical across compared policies, their computational overhead is treated as constant and excluded from comparative runtime analysis.

Across six seeds, the locked LinUCB policy achieves the highest mean F1 while maintaining the lowest operational overhead among adaptive policies. Compared with fixed-interval retraining, cumulative update runtime is reduced by 88.3%, while maintaining higher F1. Relative to threshold persistence, runtime decreases by 85.4%. Even when compared with an action-rate-matched fixed schedule, LinUCB requires 30.6% less cumulative update runtime while preserving superior predictive performance, indicating that the observed gains cannot be explained solely by updating less frequently.

Under label delays of 0–10 batches (Table II), action rate remains flat at 0.0157 with no collapse to always-update or never-update behavior. At delay 10, updates decrease (9.17 to 8.17) and F1 increases (0.2058 to 0.2229), consistent with a more conservative intervention pattern in an imbalanced setting.

Unless otherwise noted, headline values refer to the locked operating-point evaluation (Table I); metric-alignment ablation values come from dedicated 6-seed reruns and are interpreted as within-ablation contrasts.

V. RESULTS AND DISCUSSION

Airline as the primary result. Table I shows that LinUCB achieves the highest F1 among adaptive policies. The action-rate-matched baseline demonstrates that this improvement is not explained solely by updating less often, while runtime measurements confirm the same ordering under a directly observed operational metric.

Metric alignment drives behavior. Table III shows that the choice of reward metric substantially changes policy behavior under class imbalance. Optimizing ΔAccuracy causes LinUCB to over-intervene (168.83 updates) and yields lower F1 (0.1813), since majority-class accuracy dominates the reward signal. Optimizing ΔF1 instead restores conservative behavior (8.33 updates) and improves minority-class performance (F1 0.2290). This contrast confirms that DARE's orchestration is metric-agnostic, but the resulting policy behavior is shaped entirely by the chosen reward objective. (Absolute values in this ablation come from dedicated 6-seed reruns; the comparison is a within-table contrast between reward definitions, not against the main Table I results.)

A complementary reward-mode ablation (raw vs. rolling) shows that rolling smoothing reduces reward variance (0.1698 to 0.0952) while leaving fine-tune rate unchanged (0.0600), indicating a more stable learning signal without increased intervention.

Delayed-label robustness. Table II shows stable behavior across delays of 0–10 batches: action rate is unchanged at 0.0157, cost changes modestly (13.75 to 12.25), and there is no

collapse to degenerate always-update or never-update regimes. The F1 increase at delay 10 coincides with fewer updates (9.17 to 8.17), consistent with reduced over-intervention under delayed feedback in an imbalanced setting. Reward remains near zero across delays (from -0.0010 at delays 0–5 to 0.0015 at delay 10), indicating no instability or collapse in the delayed-feedback regime.

Elec2 as calibration evidence. Elec2 exhibits the same directional behavior as Airline: heuristic policies intervene substantially more frequently than LinUCB, supporting the consistency of the learned policy across temporally ordered streams. Because Elec2 is used as calibration rather than the headline benchmark, we report it as supporting evidence rather than as the primary policy comparison.

Limitations. Several limitations bound the current results. Runtime measurements were collected under a single hardware configuration. Although cumulative runtime provides a directly observed measure of operational overhead, absolute execution time may vary across deployment environments. The offline replay protocol does not capture queue lag, distributed failure modes, or live serving effects. In addition, the locked LinUCB action space in the main result is binary (defer vs. fine-tune), while broader action sets under explicit budget constraints remain future work. The evaluation uses two temporally ordered tabular datasets; generalization to image streams, text streams, or high-dimensional feature spaces with different drift characteristics remains an open question. Fairness and energy signals are logged and reported for diagnostics, but they are not enforced as primary optimization objectives in the main Airline result. Finally, the six-seed evaluation, while stronger than single-run reporting, does not yet provide formal regret bounds or convergence guarantees for the conservative operating regime.

VI. Conclusion

This paper reframes model retraining under drift as a sequential decision problem with delayed feedback, rather than a static trigger-design problem. We presented DARE, an event-driven MLOps architecture with schema-validated events and deterministic replay that enables reproducible policy-level comparison under shared execution conditions.

Empirically, on Airline 120k, a locked conservative LinUCB policy achieves a favorable runtime–performance operating point versus heuristic retraining baselines while remaining stable under delayed labels. These results support the central design claim: policy quality depends on explicit action-cost trade-offs under delay, not drift detection alone. They also expose two methodological lessons that generalize beyond the Airline benchmark.

First, threshold-based retraining policies are not configuration-invariant across datasets: a threshold setting calibrated for Airline did not transfer reliably to Elec2 without re-tuning, indicating that heuristic trigger policies require dataset-specific recalibration for stable operation. Second, DARE is reward-metric agnostic at the orchestration level, but policy behavior is strongly shaped by the chosen reward definition, as shown in Section IV, optimizing ΔAccuracy led to over-intervention. These findings reinforce that deployment-ready retraining policies need both calibrated trigger logic and metric-task alignment.

Future work. Next steps are to couple fairness signals directly to action selection, expand evaluation across more datasets and seeds, and extend from offline replay to online validation with richer budget-aware action spaces.

References

[1] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," ACM Comput. Surv., vol. 46, no. 4, pp. 1–37, Apr. 2014, doi: 10.1145/2523813.

[2] L. Chislett, C. A. Vallejos, T. I. Cannings, and J. Liley, "AMUSE: Adaptive model updating using a simulated environment," arXiv:2412.10119, 2024. [Online]. Available: https://arxiv.org/abs/2412.10119

[3] F. Regol, L. Schwinn, K. Sprague, M. Coates, and T. Markovich, "When to retrain a machine learning model," in Proc. 42nd Int. Conf. Machine Learning (ICML), ser. PMLR, vol. 267, 2025, pp. 51369–51404. [Online]. Available: https://proceedings.mlr.press/v267/regol25a.html

[4] T. M. T. Pham, K. Premkumar, M. Naili and J. Yang, "Time to Retrain? Detecting Concept Drifts in Machine Learning Systems," 2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), Ottawa, ON, Canada, 2025, pp. 260-271, doi: 10.1109/ICSE-SEIP66354.2025.00029.

[5] B. Rank, S. Triantafyllou, D. Mandal, and G. Radanovic, "Performative reinforcement learning in gradually shifting environments," in Proc. Conf. Uncertainty in Artificial Intelligence (UAI), ser. PMLR, vol. 244, 2024. [Online]. Available: https://proceedings.mlr.press/v244/rank24a.html

[6] S. Moothedath, "Neural contextual bandits under delayed feedback constraints," arXiv:2504.12086, Apr. 2025. [Online]. Available: https://arxiv.org/abs/2504.12086

[7] S. Dong, Q. Wang, S. Sahri, T. Palpanas, and D. Srivastava, "Efficiently mitigating the impact of data drift on machine learning pipelines," Proc. VLDB Endow., vol. 17, no. 11, pp. 3072–3081, Jul. 2024, doi: 10.14778/3681954.3681984.

[8] M. Böther et al., "Modyn: Data-Centric Machine Learning Pipeline Orchestration," Proceedings of the ACM on Management of Data, vol. 3, no. 1, pp. 1–30, Feb. 2025, doi: 10.1145/3709705.

[9] E. K. Katalay et al., "A multi-criteria automated MLOps pipeline for cost-effective cloud-based classifier retraining in response to data distribution shifts," arXiv:2512.11541, Dec. 2025. [Online]. Available: https://arxiv.org/abs/2512.11541

[10] O. B. Deho et al., "Is it still fair? A comparative evaluation of fairness algorithms through the lens of covariate drift," Machine Learning, vol. 114, no. 1, Jan. 2025, doi: 10.1007/s10994-024-06698-6.

[11] N. Deininger, "Newline Delimited JSON," 2013. [Online]. Available: http://ndjson.org/

[12] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in Proc. 19th Int. Conf. World Wide Web (WWW), 2010, pp. 661–670, doi: 10.1145/1772690.1772758.

[13] U.S. Department of Transportation, Bureau of Transportation Statistics, "Airline Service Quality Performance 234 (On-Time performance data)," 2026. [Online]. Available: https://www.bts.gov/browse-statistical-products-and-data/bts-publications/airline-service-quality-performance-234-time.

[14] OpenML, "Electricity (Dataset ID: 151)," [Online]. Available: https://www.openml.org/d/151.

[15] M. Harries, "SPLICE-2 Comparative Evaluation: Electricity Pricing," School of Computer Science and Engineering, The University of New South Wales, Sydney, Australia, Tech. Rep. UNSW-CSE-TR-9905, Jul. 1999. [Online]. Available: https://cgi.cse.unsw.edu.au/~reports/papers/9905.pdf